



MINIPROYECTO CONTEO DE OBJETOS

Práctica 3-a

Práctica: “Ruidos: Gaussiano y Sal y Pimienta”

¿Cuándo modelar cada tipo?

Tipo de ruido	Características	Aplicación común
Sal y pimienta	Puntos blancos y negros aleatorios	Simulación de errores de transmisión o sensores
Gaussiano	Variaciones suaves en intensidad	Simulación de ruido electrónico o térmico

Ejemplo en python

Ejemplo de **código en Python** para aplicar tanto **ruido sal y pimienta** como **ruido gaussiano** a una imagen, usando las librerías **OpenCV**, **NumPy** y **Matplotlib**:

Código en Python

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Cargar imagen en escala de grises (usar la imagen que se te indique)
imagen = cv2.imread('tu_imagen.jpg', cv2.IMREAD_GRAYSCALE)
# -----
# Función para agregar ruido sal y pimienta
def agregar_ruido_sal_pimienta(imagen, cantidad):
    imagen_ruido = imagen.copy()
    filas, columnas = imagen.shape
    num_ruido = int(cantidad * filas * columnas)
    for _ in range(num_ruido):
        i = np.random.randint(0, columnas)
        j = np.random.randint(0, filas)
        if np.random.rand() < 0.5:
            imagen_ruido[j, i] = 0 # Pimienta (negro)
        else:
            imagen_ruido[j, i] = 255 # Sal (blanco)
    return imagen_ruido
# -----
# Función para agregar ruido gaussiano
def agregar_ruido_gaussiano(imagen, media=0, sigma=20):
    gauss = np.random.normal(media, sigma, imagen.shape).astype(np.int16)
    imagen_ruido = imagen.astype(np.int16) + gauss
    imagen_ruido = np.clip(imagen_ruido, 0, 255).astype(np.uint8)
    return imagen_ruido
# -----
# Aplicar ruido
imagen_sal_pimienta = agregar_ruido_sal_pimienta(imagen, cantidad=0.02)
imagen_gaussiana = agregar_ruido_gaussiano(imagen, media=0, sigma=20)
# -----
# Mostrar resultados
```

```
fig, axs = plt.subplots(1, 3, figsize=(12, 4))
axs[0].imshow(imagen, cmap='gray')
axs[0].set_title('Imagen original')
axs[0].axis('off')
axs[1].imshow(imagen_sal_pimienta, cmap='gray')
axs[1].set_title('Ruido sal y pimienta')
axs[1].axis('off')
axs[2].imshow(imagen_gaussiana, cmap='gray')
axs[2].set_title('Ruido gaussiano')
axs[2].axis('off')
plt.tight_layout()
plt.show()
```

¿Qué necesitas para ejecutarlo?

- Instalar las librerías si no las tienes:

```
pip install opencv-python numpy matplotlib
```

- Reemplaza 'tu_imagen.jpg' por la ruta de tu imagen.
- Experimenta usando la imágenes que se te solicitó tomaras en color, gris, binarias.